Cupcake

Rapport for 2. semester projekt

- Deltagere Klasse B:
  - Musa Sayed, cph-ms1425@stud.ek.dk, Musa-909
  - Rasull Khan Hossein, raho1000@stud.ek.dk, rasull7452
  - Khattab Elmir, cph-ke143@stud.ek.dk, KhattabE
  - Amaan Mahomed, cph-am594@stud.ek.dk, AmaanM3

Indholdsfortegnelse

Indholdsfortegnelse **1**

Indledning *2*

Baggrund 2

Teknologi valg 3

Krav *3*

User stories 4

Domæne model og ER diagram *5*

Domænemodel 5

ER diagram 7

Navigationsdiagram *9*

Særlige forhold *11*

Status på implementation *15*

Proces *16*

Indledning

Denne rapport er skrevet til en fagfælle, dvs. en anden datamatikerstuderende på 2. semester, med tilsvarende fagligt niveau, som ikke har kendskab til cupcakeopgaven i forvejen. Formålet er at introducere projektet og skabe en forståelse for både baggrunden, systemets opbygning og de vigtigste valg, der er blevet truffet undervejs.

Cupcake projektet handler om at udvikle en webshop til virksomheden Olsker Cupcakes, som er baseret på 9 forskellige user stories som vi har taget udgangspunkt i igennem udviklingen af webshoppen

Baggrund

Vores kunde er et økologisk cupcakefirma som ligger på Bornholm ved navn Olsker Cupcakes. De har sat en række krav til den webshop som vi skal udvikle for dem. Kravene til webshoppen er følgende:

- Kunden skal kunne logge ind eller oprette en profil
- Kunden skal kunne bestille en cupcake hvor der kan vælges bund og top til cupcaken
- Kunden skal kunne se hver ordrelinje i sin indkøbskurv sådan at den samlede pris kan kunne ses
- Når kunden er logget ind på hjemmesiden, skal deres mail være synlig på hver side
- Kunden skal kunne fjerne ordrelinjerne i sin indkøbskurv
- Administratorer skal kunne indsætte et beløb inde på den enkelte kundes profil, sådan at kunden kan købe cupcakes med det indsatte beløb
- Administratorer skal kunne se alle ordrer i systemet
- Administratorer skal kunne se alle kunderne i systemet
- Administratorer skal kunne fjerne en ordre fra systemet

Teknologi valg

- IDEA intellij 2025.2.6.1
- Java 21
- JDBC
- PostgreSQL 42.7.10
- HikariCP ver. 5.1.0
- Javalin 7.1.0
- Thymeleaf 3.1.3. RELEASE
- Slf4j-simple 2.0.17

Krav

Olsker Cupcakes ønsker et system, som gør det nemmere for kunder at bestille cupcakes online, samtidig med at virksomheden får et bedre overblik over brugere og ordrer. Visionen med systemet er at skabe en digital bestillingsproces, hvor kunder selv kan oprette en konto, logge ind og sammensætte cupcakes med de valgfri bunde og toppings, der er tilgængelige. Herefter kan de lægge deres valg i en kurv og gennemføre en ordre.

Systemet skaber værdi for virksomheden, fordi bestillinger bliver samlet et sted, gemt i databasen og kan tilgås af administratoren. Det gør det lettere at holde styr på kunder, ordrer og betalinger, samtidig med at kunden får en mere enkel og overskuelig måde at bestille cupcakes på.

User stories

- US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken i Olsker og hente min ordre.

- US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.

- US-3: Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.

- US-4: Som kunde kan jeg se mine valgte ordrelinier en indkøbskurv, så jeg kan se den samlede pris.

- US-5: Som kunde eller administrator kan jeg logge på systemet med e-mail og kodeord. Når jeg er logget på, skal jeg kunne se min e-mail på hver side (evt. i topmenuen, som vist på mockup'en).

- US-6: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

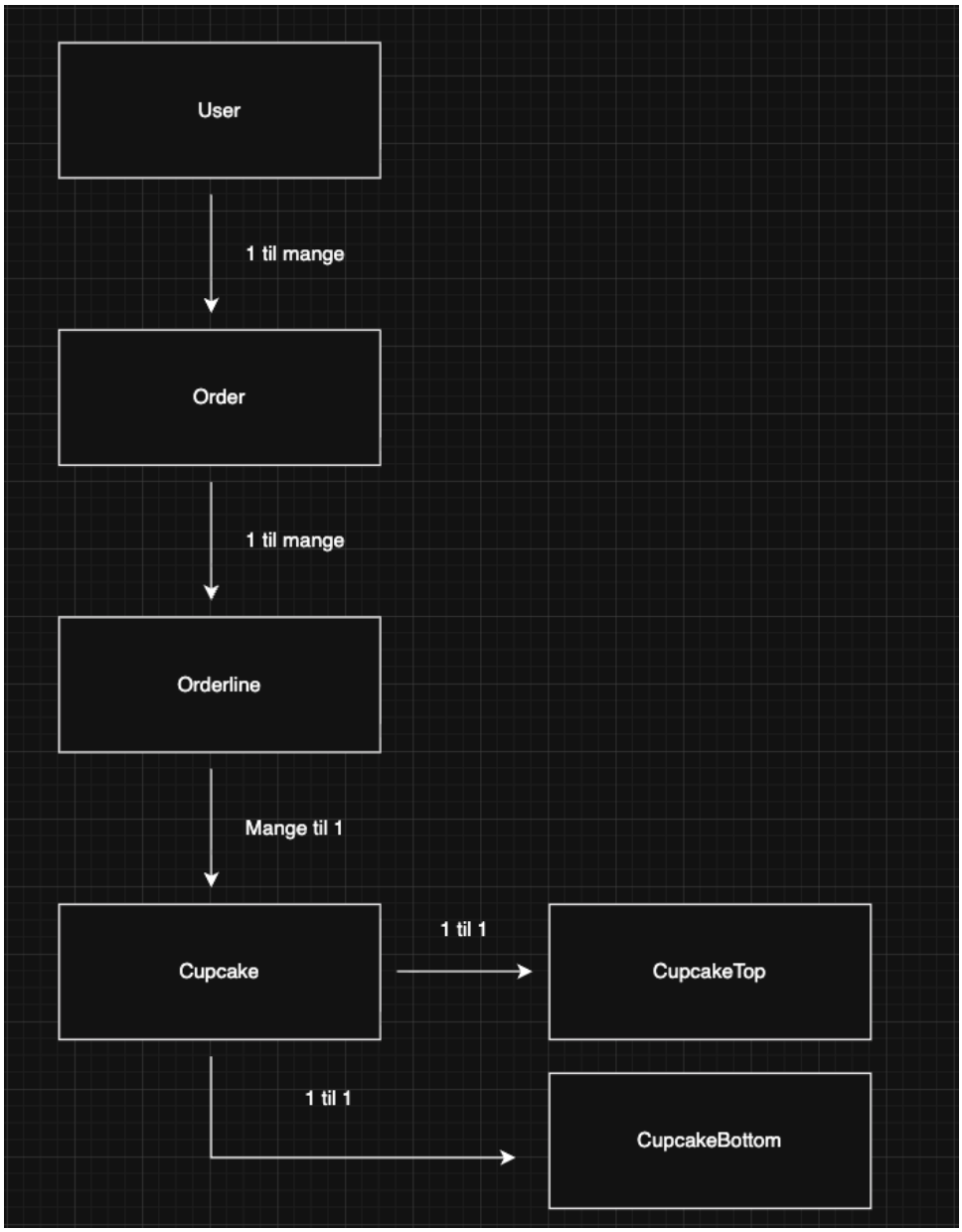
- US-7: Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

- US-8: Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.

- US-9: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer. F.eks. hvis kunden aldrig har betalt.

Domæne model og ER diagram

Domænemodel



Figur 1: Domænemodel over systemets centrale klasser og relationer.

Diagrammet viser de vigtigste klasser i systemet: *User, Order, Orderline, Cupcake, CupcakeTop og CupcakeBottom*. Fokus er på relationer og hvordan klasserne hænger sammen, ikke på metoder eller alle attributter.

- En User har en 1-til-mange relation til Order. Det betyder, at én bruger kan have flere ordrer, men en ordre tilhører kun én bruger.

- En Order har en 1-til-mange relation til Orderline, da en ordre kan bestå af flere linjer.

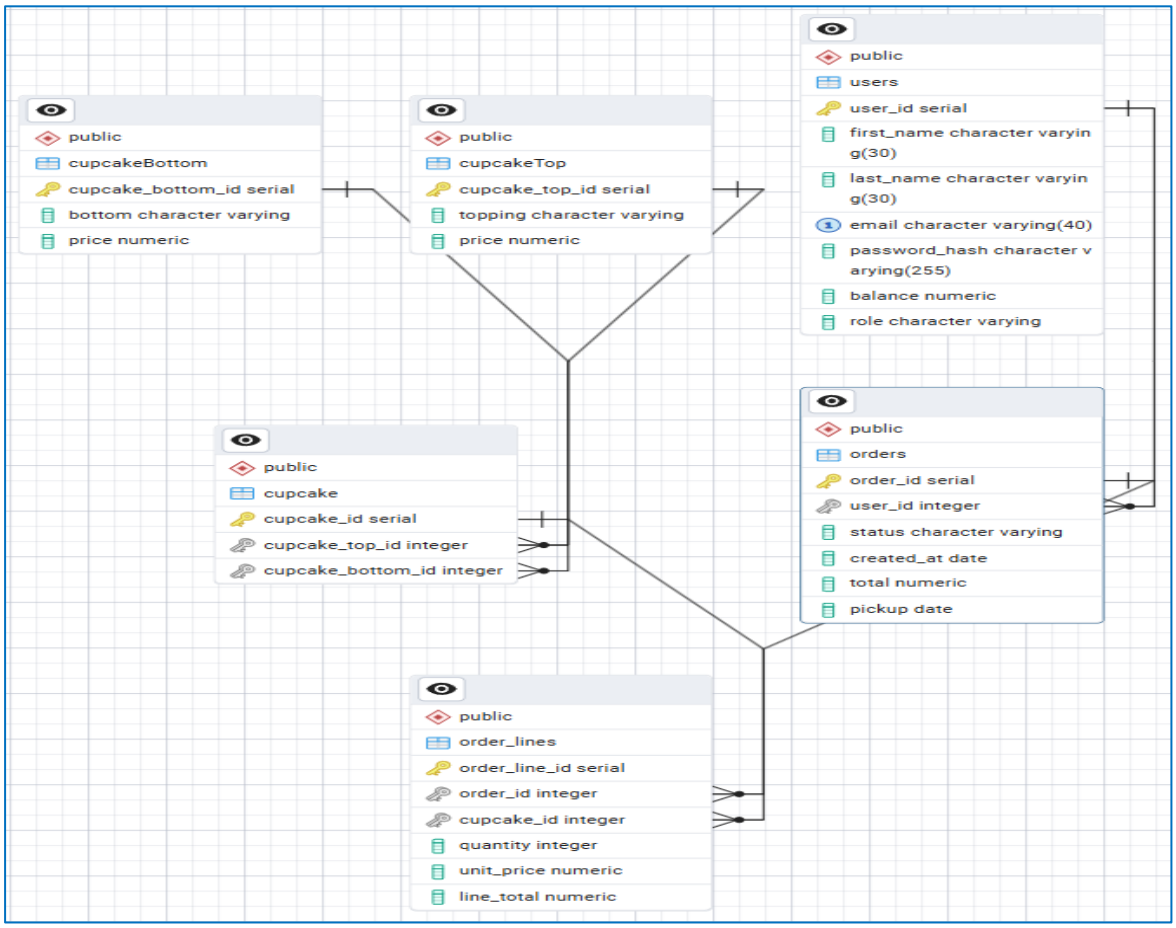
- En Orderline har en mange-til-1 relation til Cupcake, fordi flere ordrelinjer kan referere til den samme cupcake.

- En Cupcake har en 1-til-1 relation til både CupcakeTop og CupcakeBottom, da hver cupcake består af én topping og én bund.

En vigtig beslutning var at bruge Orderline mellem Order og Cupcake. Man kunne have lavet en direkte relation mellem Order og Cupcake, men så ville det være svært at gemme ekstra information som antal og pris. Ved at bruge Orderline kan man gemme disse oplysninger, og det gør modellen mere fleksibel.

Cupcake er delt op i CupcakeTop og CupcakeBottom for at undgå gentagelser i databasen. Det gør det også muligt at genbruge toppings og bunde på tværs af forskellige cupcakes. Relationerne her er lavet som 1-til-1, fordi en cupcake altid har én topping og én bund.

ER diagram



Figur 2: ER-diagram over databasen.

ERD'en giver os et overblik over en database, som håndterer brugere, ordrer samt cupcakes. Modellen består af tabellerne *users, orders, order_lines, cupcake, cupcakeTop og cupcakeBottom*. En bruger kan have flere ordrer, og hver ordre kan indeholde flere ordrelinjer, som hver repræsenterer en bestemt cupcake og en bestemt mængde. En cupcake består af en bund og en topping, som er modelleret i separate tabeller.

Der er anvendt fremmednøgler til at sikre relationerne mellem tabellerne:

- orders.user_id refererer til users.user_id
- order_lines.order_id refererer til orders.order_id
- order_lines.cupcake_id refererer til cupcake.cupcake_id
- cupcake.cupcake_top_id refererer til cupcakeTop.cupcake_top_id
- cupcake.cupcake_bottom_id refererer til cupcakeBottom.cupcake_bottom_id

Relationen mellem orders og cupcake er en mange-mange relation, som er implementeret via koblingstabellen *order_lines*. Denne tabel indeholder ikke kun fremmednøgler, men også ekstra attributter som *quantity, unit_price* og *line_total,* hvilket gør det muligt at gemme detaljer om hver ordrelinje.

Alle tabeller anvender automatisk genererede ID'er som primærnøgler. Dette gør det nemt at identificere rækker unikt og sikrer enkel håndtering af relationer.

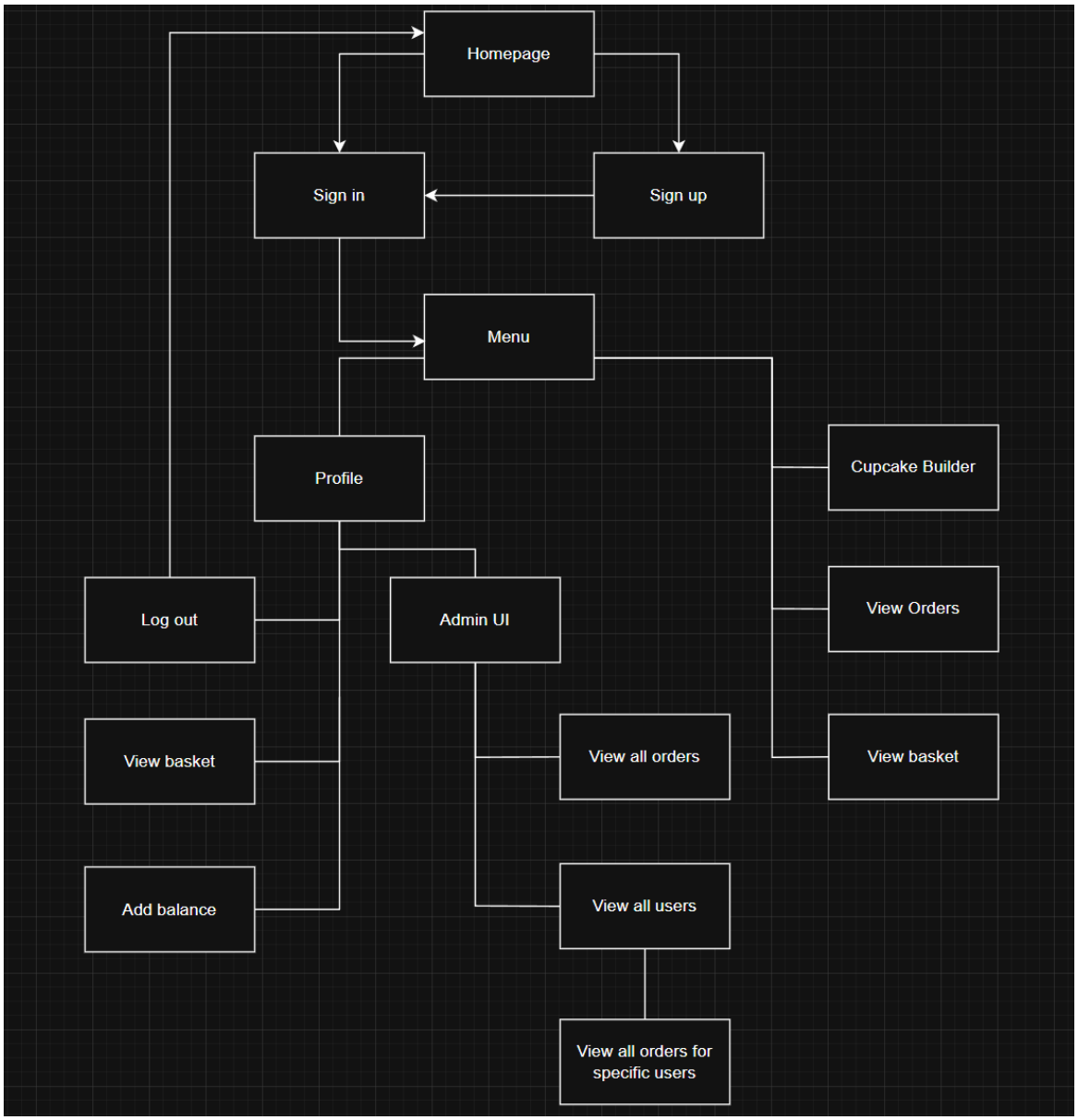
Modellen er designet med fokus på fleksibilitet og dataintegritet:

- Cupcakes er opdelt i top og bottom for at undgå redundans og gøre systemet mere fleksibelt
- order_lines gør det muligt at gemme flere produkter pr. ordre
- Priser gemmes direkte i ordrelinjer for at sikre historisk korrekt data

Der er ikke anvendt 1-1 relationer, da alle entiteter har en selvstændig betydning i systemet.

Der er anvendt constraints som UNIQUE på *users.email* for at sikre, at hver bruger har en unik emailadresse.

Navigationsdiagram



Figur 3: Navigationsdiagram over systemets sider og brugerflow. Viser navigation mellem centrale sider som login, menu, profil, cupcake builder og indkøbskurv.

Det, som brugeren oplever, er en række websider, hvor man kan indtaste oplysninger og navigere videre til andre sider. I større systemer kan det være svært at bevare overblikket over, hvilke sider der eksisterer, og hvordan man navigerer imellem dem. Navigationsdiagrammet er beregnet på at vise dette på en mere overskuelig måde. Som led i beskrivelsen af navigationsdiagrammet skal følgende med:

- Oversigtsdiagrammet. Hvis det bliver for stort, må man dele det op. Men det er vigtigt at der er et overordnet diagram.
- Hvis man har benyttet sig af en “fælles navigations bar” i toppen af alle sider, skal man forklare det.
- Hvis nogle sider kun kan nåes af nogle brugere (dem der har konto, dem der er logget ind, dem der arbejder i butikken,...), så skal det fremgå.

Navigationsdiagrammer laves som UML Tilstandsmaskiner state machine diagram. Bemærk specielt at det, der hedder “composite state”, er meget velegnet til at indkapsle at man er logget ind.

Særlige forhold

- **Hvilke informationer gemmes i session**
  - De informationer, der bliver gemt i en session, omfatter blandt andet brugeren, når man er logget ind (så man ikke behøver at logge ind igen, når man navigerer til en anden side på websiden). Derudover bliver indkøbskurven også gemt i sessionen.

Nedenstående figurer viser, hvordan henholdsvis brugeren og indkøbskurven gemmens i sessionen

```
    ctx.sessionAttribute("currentUser", user);
    ctx.redirect( location: "/menu");
}
```

Figur 4: Gemning af bruger i session. Viser hvordan den loggede bruger gemmes i sessionen for at bevare loginstatus.

```
List<BasketItem> basket = ctx.sessionAttribute( key: "basket");
if (basket == null) {
    basket = new ArrayList<>();
}

basket.add(item);
ctx.sessionAttribute("basket", basket);
```

Figur 5: Håndtering af indkøbskurv i session. Viser hvordan indkøbskurven oprettes og gemmes i sessionen.

- **Hvordan håndterer man exceptions. Det kommer vi tilbage til senere i semesteret.**
  - Exceptions håndteres her med try-catch blokke. Koden forsøger at udføre databasekaldet i try, og hvis der opstår en SQLException, bliver den grebet i catch-blokken. Lige nu bliver fejlen kun printet med System.out.println, men en bedre løsning senere vil være at logge fejlen eller kaste en mere passende exception videre.

```java
// creates a new user in the database
public void createUser(User ! user) {
    String sql = """
        INSERT INTO users (first_name, last_name, email, password_hash, balance, role) VALUES (?,?,?,?,?,?)
        """;

    try (
            Connection connection = connectionPool.getConnection();
            PreparedStatemen preparedStatement = connection.prepareStatement(sql)){

        preparedStatement.setString( parameterIndex: 1, user.getFirstName());
        preparedStatement.setString( parameterIndex: 2, user.getLastName());
        preparedStatement.setString( parameterIndex: 3, user.getEmail());
        preparedStatement.setString( parameterIndex: 4, user.getPasswordHash());
        preparedStatement.setDouble( parameterIndex: 5, user.getBalance());
        preparedStatement.setString( parameterIndex: 6, user.getRole());

        preparedStatement.executeUpdate();

    } catch (SQLException e){
        System.out.println("An error has has happend" + sql );
    }
}
```

Figur 6: Eksempel på håndtering af exceptions ved et databasekald

- **Hvordan man har valgt at lave validering af brugerinput.**
  - Valideringen er implementeret som simpel server-side validering. Input hentes fra ctx.formParam, hvorefter der laves checks som fx password match og om brugeren allerede eksisterer. Derudover bruges databasen til at validere login-oplysninger. Hvis valideringen fejler, returneres en fejlbesked til brugeren.

Nedenstående figur viser et eksempel på, hvordan validering af brugerinput er implementeret i systemet.

```
public static void handleSignUp(Context ! ctx) {
    String firstName = ctx.formParam( key: "firstName");
    String lastName = ctx.formParam( key: "lastName");
    String email = ctx.formParam( key: "email");
    String password = ctx.formParam( key: "password");
    String confirmPassword = ctx.formParam( key: "confirmPassword");

    if (!password.equals(confirmPassword)) {
        ctx.result( resultString: "Passwords do not match");
        return;
    }

    UserMapper userMapper = new UserMapper(ConnectionPool.getInstance());

    if (userMapper.getUserByEmail(email) != null) {
        ctx.result( resultString: "User already exists");
        return;
    }

    User user = new User( userId: 0, firstName, lastName, email, password, balance: 500.00, role: "user");
    userMapper.createUser(user);

    ctx.redirect( location: "/signin");
}
```

Figur 7: Validering ved oprettelse af bruger (sign up)

```
public static void handleSignIn(Context ! ctx) {
    String email = ctx.formParam( key: "email");
    String password = ctx.formParam( key: "password");

    UserMapper userMapper = new UserMapper(ConnectionPool.getInstance());
    User user = userMapper.validateLogin(email, password);

    if (user == null) {
        ctx.attribute("error", "Wrong email or password. Try again.");
        ctx.attribute("email", email);
        ctx.render( filePath: "signin.html");
        return;
    }

    ctx.sessionAttribute("currentUser", user);
    ctx.redirect( location: "/menu");
}
```

Figur 8: Validering ved login (sign in). Viser valisering af login oplysninger mod databasen samt håndtering af fejl og succesfuldt login

- **Hvilke brugertyper, der er valgt i databasen, og hvordan de er brugt i jdbc**
  - I databasen er der valgt to brugertyper: user og admin, som er gemt i kolonnen role i tabellen users. Rollen er gemt som en teksttype (varchar). I JDBC bliver rollen hentet fra databasen med ResultSet ved brug af rs.getString("role") og gemt i et User-objekt. På den måde kan applikationen bruge rollen til at styre adgang til forskellige funktioner, fx almindelige sider for brugere og admin-sider for administratorer

```
    String role = rs.getString( columnLabel: "role");

    return new User(id, firstName, lastName, userEmail, passwordHash, balance, role);
}
```

Figur 9: Hentning af brugerrolle fra databasen i JDBC, ved brug af ResultSet og gemmes i et User-objekt

```
if ("admin".equalsIgnoreCase(currentUser.getRole())) {
    ctx.redirect( location: "/admin-profile");
    return;
}
```

Figur 10: Anvendelse af brugerrolle til styring af adgang, eksempelvis ved at omdirigere en administrator til en specifik side

Status på implementation

Dette afsnit skal liste hvor langt man er nået med implementationen. Typiske ting man kan have sprunget over er:

- **Man har ikke nået at lave alle de websider man har med i navigationsdiagrammet.**

Det har ikke været muligt at implementere alle de websider, som er vist i navigationsdiagrammet. Specifikt mangler siden til "Add balance", som derfor ikke er implementeret i systemet. Funktionaliteten er planlagt og indgår i designet, men er udeladt på grund af tidsbegrænsning

- **Man har fundet en fejl "i sidste øjeblik", men har ikke haft tid til at rette det. - F.eks. at man har brugt session forkert, sådan at man på en af siderne kan komme ind uden at være logget ind.**

Hvis man prøver på at købe en cupcake, men ikke har nok penge i saldoen til det så kommer der bare en sort blank side hvor der står at man ikke har nok balance. Det er meningen at man forbliver på samme side og at der bare dukker en tekst op som siger at man ikke har nok penge i saldoen til at fuldføre købet.

```
if (currentUser.getBalance() < total) {
    ctx.result( resultString: "Not enough balance");
    return;
}
```

Figur 11: Håndtering af utilstrækkelig saldo ved køb hvor der returneres en fejlbesked

Proces

I starten af projektet var planen, at vi ville arbejde struktureret med opgaven, ved blandt andet at dele user stories op mellem os og løbende samle delene til et fælles funktionelt system. Vi havde også planlagt at bruge branches, når vi kodede, så der ikke opstod merge konflikter, samt at arbejde med de forskellige teamroller såsom Scrum Master, Product Owner og Tech Lead på skift gennem projektforløbet. Tanken bag det var, at det skulle skabe struktur og gøre det tydeligt, hvem der havde ansvaret for hvad gennem de forskellige faser.

I praksis kom vores projekt dog til at være mere fælles, end vi havde regnet med. Vi sad oftest sammen og udviklede store dele af koden i fællesskab, blandt andet gennem Discord. Det havde dog også en positiv påvirkning på arbejdet, da man hurtigt kunne hjælpe hinanden med at løse de problemer, man stødte på og sad fast i. Vi brugte branches, men der var også gange, hvor vi glemte det, hvilket vi skal arbejde på næste gang.

Når det kom til rollerne i selve teamet, havde vi i starten en plan om at skifte mellem de forskellige teamroller undervejs i projektet. Det blev dog ikke implementeret i praksis, da vores fokus ikke i særlig høj grad lå på at få dem fordelt og fulgt konsekvent. I princippet var vi derfor alle lidt det hele på samme tid, da ingen havde en helt fast rolle såsom Scrum Master. Derfor kom vores samarbejde også i højere grad til at bygge på fælles beslutninger og løbende dialog frem for en fast rollefordeling.

Noget af det, der gik godt, var, at vi fik lavet store dele af systemets funktioner og bygget løsninger, hvor frontend, backend samt database hænger sammen. Det gode ved, at vi arbejdede meget sammen, var, at det gav os en fælles forståelse for projektet og gjorde det lettere at debugge de fejl, som opstod undervejs.

Der var dog også punkter, som kunne have været bedre. Selvom vi brugte branches, var vi ikke konsekvente med det gennem hele projektet. Vi kunne også have været bedre til at holde fast i de teamroller, som vi i starten havde tildelt, da det muligvis kunne have

givet et tydeligere overblik over ansvar og arbejdsopgaver. Vi kunne desuden have været bedre til at planlægge tidligere i forløbet.

Noget af det vigtigste, vi har lært af processen, er, at en tydelig struktur og en konsekvent arbejdsmetode gør samarbejdet lettere. Næste gang vil vi i højere grad sørge for at bruge branches mere konsekvent, fordele roller tydeligere og følge en mere fast plan undervejs. Samtidig vil vi stadig gerne arbejde fælles, da det fungerede godt, når vi skulle løse større problemer sammen.

I fremtiden vil vi gerne gøre kundernes brugere mere sikker ved at enkryptere deres kode når det bliver sendt fra bruger til database sådan at hvis vores database skulle blive komprimeret så ville der kun være enkrypteret versioner af alle koder i databasen og bruger sikkerheden ville vedligeholdes. Dette kan man opnå ved b.la. at gøre brug af JBCrypt som er en Maven dependency som kan enkryptere bruger inputs.